

Architecture

State Management

State is split across three layers depending on scope and persistence:

State	Where it lives	Why
<code>activeCall</code>	Zustand (<code>useBirdSpawnStore</code>)	Base and Identification both need it
<code>user, session</code>	Zustand (<code>useAuthStore</code>)	Needed across the whole app
<code>base, progress</code>	Zustand (<code>useGameStore</code>)	Persisted, shared across pages
<code>hintsRevealed, result</code>	<code>useState</code> in <code>useIdentificationSession</code>	Local to one session, resets on finish
<code>selected</code> biome	<code>useState</code> in <code>WorldMap</code>	Local UI toggle, nothing else needs it
<code>username, password</code>	<code>useState</code> in <code>LoginForm</code>	Controlled inputs, thrown away after submit
spawn interval ID	<code>useRef</code> in <code>useBirdSpawner</code>	Needs to persist but must not trigger re-renders

General rule: ephemeral UI state lives in `useState`, timers and DOM refs in `useRef`, anything shared across pages in Zustand.

Component Structure

`Base.tsx` is the main game screen — it composes three focused child components:

```
<BirdStage />           // renders the animated birds
<ProgressSidebar />    // shows score and discovered birds
<ActiveCallBanner /> // handles the call notification + countdown
```

`Identification.tsx` swaps between the guess UI and the result UI:

```
{!result && <HintSystem ... />}
{!result && <BirdSearchInput ... />}
{result && <ResultFeedback ... />}
```

`App.tsx` uses guard components with early returns for auth-protected routes:

```
function AuthGuard({ children }) {
  const user = useAuthStore(s => s.user)
  if (!user) return <Navigate to="/" replace />
  return children
}
```

```
    return children
  }
```

Data Flow

HintSystem receives data and a callback from **Identification.tsx** — it has no direct store access:

```
<HintSystem
  bird={targetBird}
  hintsRevealed={session.hintsRevealed}
  onReveal={session.revealHint}
/>
```

ActiveCallBanner receives a navigation callback from **Base** — it does not know what **onListen** does:

```
<ActiveCallBanner onListen={() => navigate('/app/identify')} />
```

Navbar skips props entirely and reads directly from Zustand:

```
<Navbar /> // gets user, progress, base all from stores
```

Custom Hooks

useIdentificationSession — bundles the entire quiz session (target bird, hints, attempts, result) so **Identification.tsx** stays declarative:

```
const { targetBird, result, hintsRevealed, submitGuess, giveUp, finish } =
useIdentificationSession()
```

useBirdSpawner — encapsulates the game loop (spawning, calling, cleanup). One line in **Base.tsx** starts everything:

```
useBirdSpawner(base?.biomeId)
```

It uses a **useRef** for the interval ID so updating it does not trigger a re-render and restart the loop:

```
const spawnTimerRef = useRef<ReturnType<typeof setInterval> | null>(null)
spawnTimerRef.current = setInterval(trySpawn, baseInterval)
```

Key Effects

App hydration — loads game state from localStorage after login:

```
useEffect(() => {  
  if (user) hydrate(user.id)  
}, [user?.id])
```

Game loop lifecycle — starts on mount, cleans up on unmount:

```
useEffect(() => {  
  spawnTimerRef.current = setInterval(trySpawn, baseInterval)  
  return () => {  
    clearInterval(spawnTimerRef.current)  
    clearSpawns()  
  }  
}, [biomeId])
```

Call countdown — resets whenever a new call starts:

```
useEffect(() => {  
  if (!activeCall) return  
  setElapsed(0)  
  const interval = setInterval(() => { /* tick */ }, 100)  
  return () => clearInterval(interval)  
}, [activeCall?.instanceId])
```